What went well

1. Model Design
    a. Every models follows a consistent pattern
    b. Table and Column Attributes keep the DB schema explicit unreadable my code

```
[Table("users")]
public class User
{
    [Key]
    [Column("id")]
    public int Id { get; set; }

    [Required]
    [MaxLength(150)]
    [Column("email")]
    public string Email { get; set; } = string.Empty;

    [Column("is_active")]
    public bool IsActive { get; set; } = true;

    // Navigation properties properly initialized
    public ICollection<Bus> Buses { get; set; } = new List<Bus>();
    public ICollection<Booking> Bookings { get; set; } = new
List<Booking>();
}
```

2. Interface based services
    a. Dependency Injection
    b. Loose Coupling
    c. Here we can able to detach the functions and more reliable because the each interfaces provides the Separate implementation

```
public interface IAuthService
{
    Task<AuthResponseDto> RegisterAsync(RegisterDto dto);
    Task<LoginAsync> LoginAsync(LoginDto dto);
    Task<UserProfileDto> GetProfileAsync(int userId);
    Task<UserProfileDto> UpdateProfileAsync(int userId,
UpdateProfileDto dto);
}
```

3. Transaction safety in booking service
    a. here I am using the serializable transaction this transaction have the highest isolation level so this is correctly prevents double booking a seat
4. Structured DTO Folders
    a. Here im using the DTO folders based on the domains
    b. To organize the files

```
AdminController.cs (412 lines)
 ├── Operator management (approve, reject, block, unblock)
 ├── Booking management
 ├── Revenue reports (3 different report endpoints)
 └── Dashboard + Audit logs
```

5. How I used the Partial classes

```
// AdminController.cs — base definition
[ApiController]
[Route("api/admin")]
[Authorize(Roles = "admin")]
public partial class AdminController : ControllerBase
{
    private readonly AppDbContext _db;
    private readonly IEmailService _emailService;
    private readonly ILogger<AdminController> _logger;
    private readonly IRouteService _routeService;

    public AdminController(AppDbContext db, IEmailService
emailService,
        ILogger<AdminController> logger, IRouteService routeService)
    { _db = db; _emailService = emailService; _logger = logger;
_routeService = routeService; }

    private async Task LogActionAsync(...) { ... }
}
```

```
// AdminController.Reports.cs — partial, same class, different file
public partial class AdminController
{
    [HttpGet("reports/daily")]
    public async Task<IActionResult> DailyReport([FromQuery]
DateTime? date) { ... }

    [HttpGet("reports/revenue-by-operator")]
    public async Task<IActionResult> RevenueByOperator() { ... }
}
```

Here in my project I didn’t used the Abstract classes anymore

Okay many of the time like when we using the interface and we definitely want to give a proper definition to the specific function right so but when we using the abstract classes we can able to define the functions so whenever we need we can able to implement but in the interface whenever we using like implement the interface we definitely want to like give the definition for the function so like this its very like when we know when we don't know about the anyone of the function we can skip the function right we can skip the definition of the function so using the abstract classes

Without use of Abstract class

```
// In User.cs
[Key] [Column("id")] public int Id { get; set; }
[Column("created_at")] public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

// In Booking.cs — SAME thing again
[Key] [Column("id")] public int Id { get; set; }
[Column("booked_at")] public DateTime BookedAt { get; set; } =
DateTime.UtcNow;

// In Bus.cs — SAME thing again
[Key] [Column("id")] public int Id { get; set; }
[Column("created_at")] public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
```

With Use of Abstract class

Here we can able to implement the need of user cs booking dot cs bus dot cs we can use as a template for the base entity so then we can able to derive the things based on our needs

```
// Models/BaseEntity.cs — abstract base
public abstract class BaseEntity
{
    [Key]
    [Column("id")]
    public int Id { get; set; }

    [Column("created_at")]
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Not went Well

1. Abstract Class
    a. We can create a base class (like BaseEntity) to store common properties such as Id and CreatedAt. Other classes like Account or User can inherit from it instead of repeating the same code.
2. Repository pattern
    a. The repository handles all CRUD operations. It keeps data logic separate from the rest of the application and makes the system easier to manage.
3. Service Layer
    a. The service layer handles business logic. It connects the repository and other features like notifications without mixing responsibilities.